

Genetic Algorithm

CSC3034 Computational Intelligence

Dr Tang Tiong Yew

Fundamental ideas of genetic algorithm

Structure of a genetic algorithm

Selection

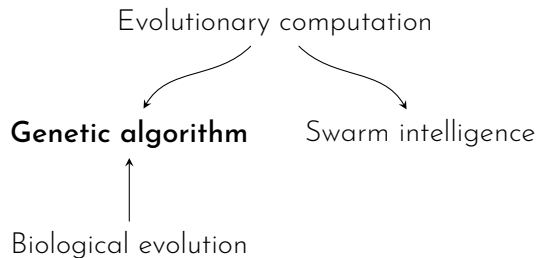
Problem formulation

Example

Evolution

Variants

Evolutionary computation



Evolution is

an optimisation process to improve the ability of an organism (or system) to survive in dynamically changing and competitive environments.

Charles Darwin developed the theory of **natural selection** which became the foundation of biological evolution.

There are two parts to the Darwinian theory of evolution, *reproduction* and *mutation*.

First part states that,

- ▶ this is a world with limited resources and stable populations.
- ▶ Each individual competes with others for survival.
- ▶ Individuals with **good** characteristics are more likely to survive and reproduce.
- ▶ Characteristics are passed on to their offsprings.
- ▶ Over time, these good characteristics become dominant among the population.

There are two parts to the Darwinian theory of evolution, *reproduction* and *mutation*.

The second part states that,

- ▶ during production of a child organism, **random events** cause **random changes** to the child organism's characteristics (mutation).
- ▶ If these new characteristics are beneficial to the organism, i.e. good characteristics, then
 - the chance of survival increases,
 - the chance of reproduction increases.
- ▶ These new characteristics are more likely to become dominant in the population.

What if an organism mutated and developed worse characteristics?

These processes (**natural selection**, **survival of the fittest**, **reproduction**, and **mutation**) form the fundamental components for the genetic algorithm.

- ▶ Each member in a population is considered as an individual **chromosome**.
- ▶ The characteristics of an individual are encoded as **genes** in the chromosome.

Generic genetic algorithm

Genetic algorithm is

an optimisation process of which the purpose is to identify the chromosome with the best combination of genes.

What is an optimisation problem?

- ▶ finding the value that will maximise/minimise another value(s)

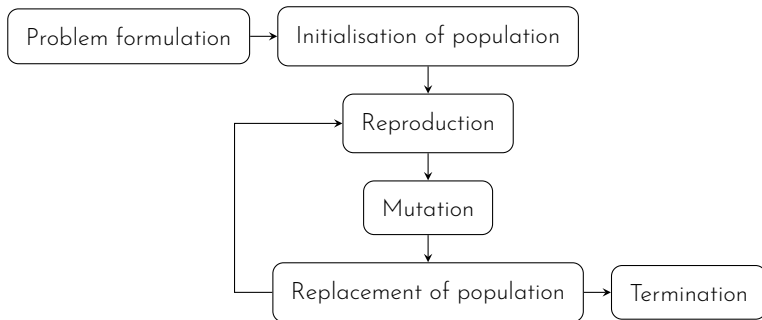
Examples,

- ▶ purchasing a laptop - optimise the combination of specifications to maximise the performance - to - money ratio
- ▶ going from one place to another place - optimise the mode transportation and route to take to minimise the time/cost
- ▶ choosing the font size of a book you are writing - optimise the font size to maximise the readability and minimise the cost of printing
- ▶ any other example you can think of?
- ▶ are search problems optimisation problems?

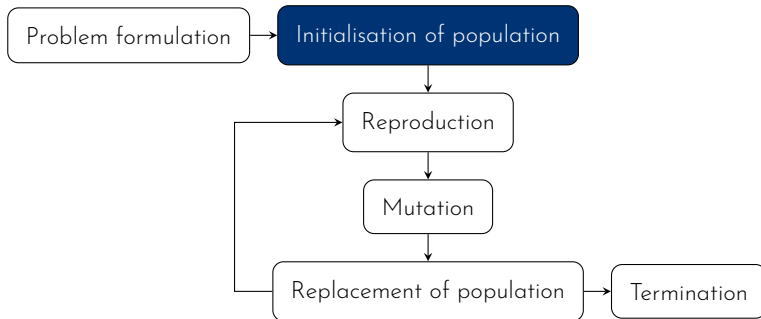
Genetic algorithm is an algorithm to solve an optimisation problem in which we are

- ▶ optimising the values of the chromosome
- ▶ to maximise the fitness of the chromosome

Genetic algorithm



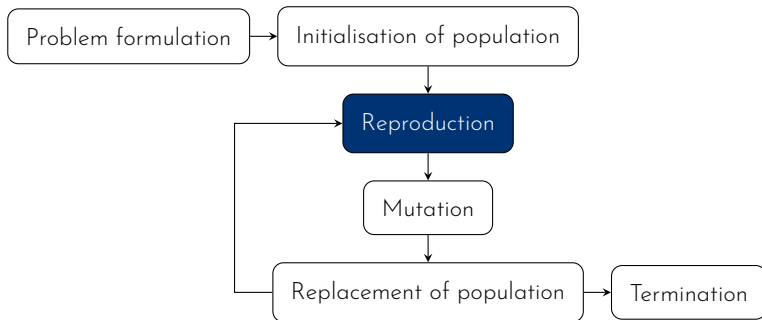
Genetic algorithm



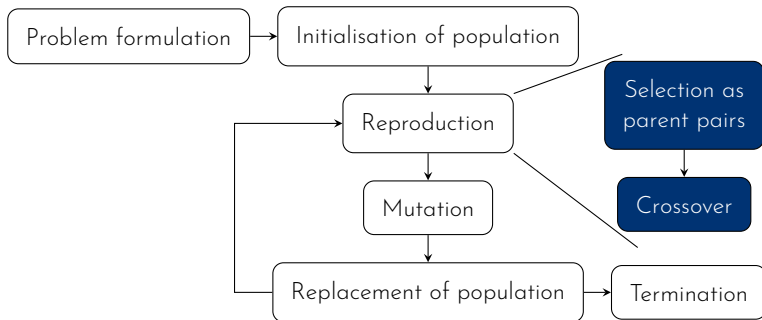
An initial population is generated with n number of chromosomes with random value assigned to the genes.

- ▶ n - the size of the initial population.
Larger size increases diversity, thus improves the exploration abilities of the population, but higher computational complexity per generation. Smaller size provides a small part of the possible solutions, shorter time for computation per generation, but more generations to converge to the optimal solution.
- ▶ **random value**
to make sure that the initial population uniformly represented the entire search space.

Genetic algorithm

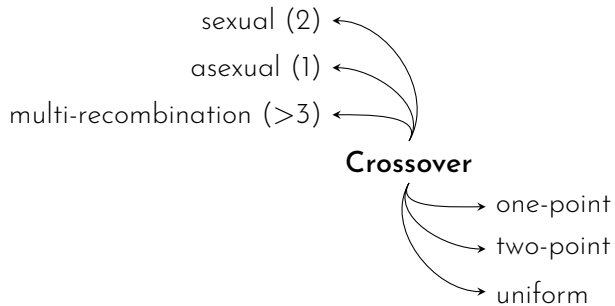


Genetic algorithm

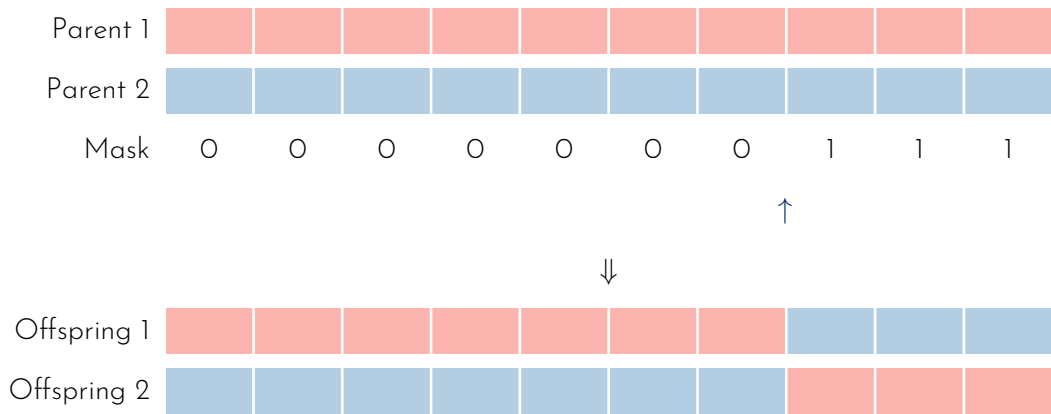


Reproduction

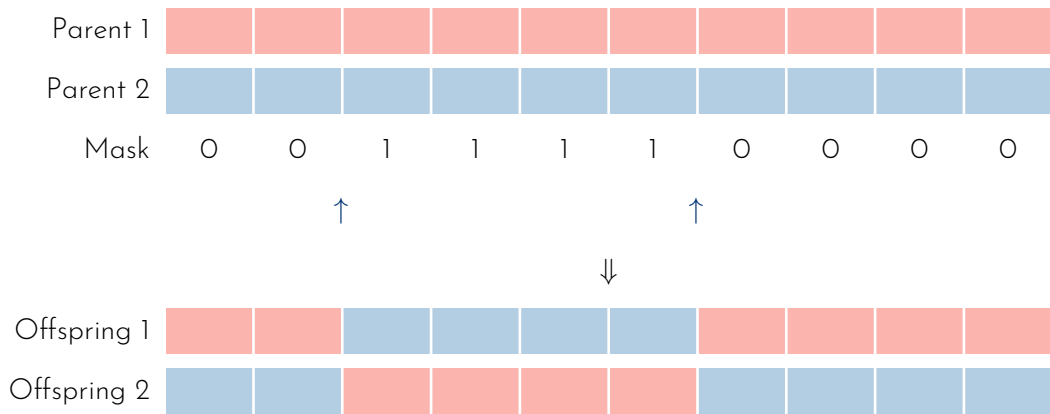
Crossover



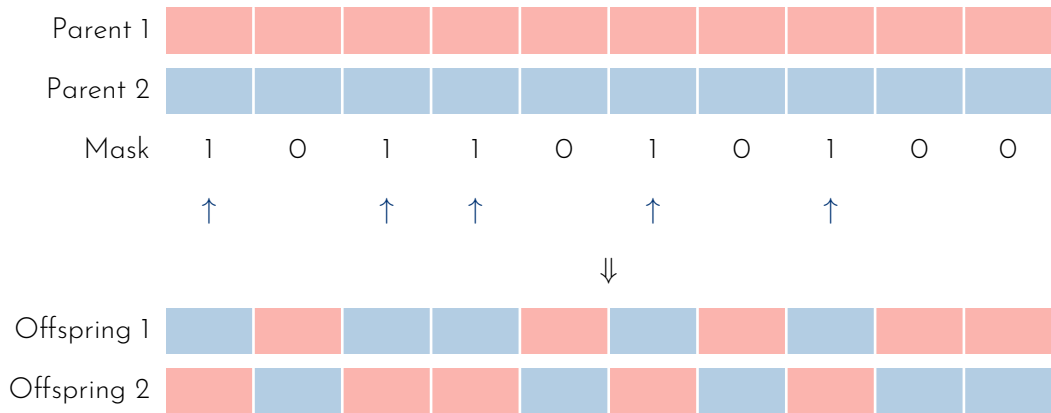
One-point crossover



Two-point crossover



Uniform crossover

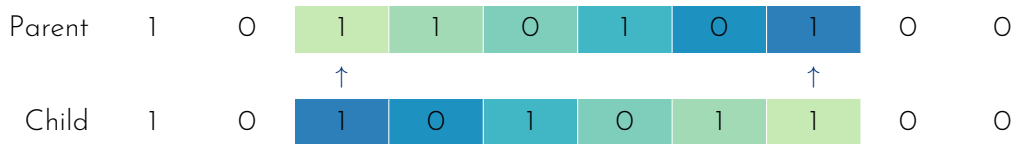


How about asexual or multi-recombination?

Crossover

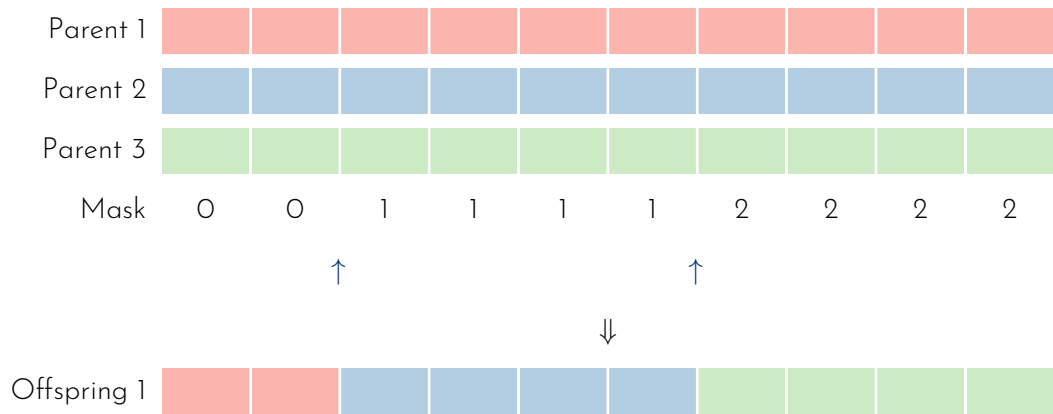
Asexual crossover

inversion – switching end to end of a portion of the parent

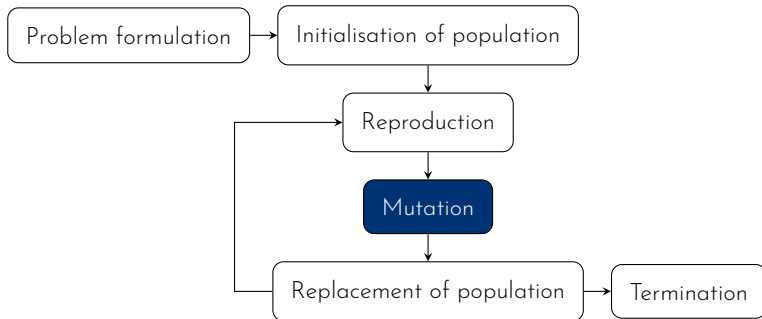


Multi-recombination crossover

n -point crossover - with n parents, $n - 1$ crossover points are selected



Genetic algorithm



Mutation aims to introduce new genetic material into an existing individual, thus increases the diversity of the population.

The mutation probability/mutation rate, p_m is usually small.

Mutation is implemented using

- ▶ **uniform** (random) mutation and
- ▶ **inorder** mutation.

Uniform mutation

All bits have the equal mutation probability p_m .

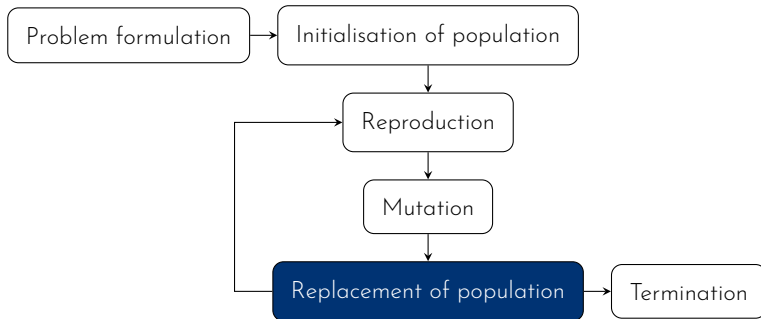
Before mutation	0	1	1	0	1	1	1	0	0	0
mutation				↑	↑					↑
After mutation	0	1	1	1	0	1	1	0	0	1

Inorder mutation

Two mutation points are randomly selected. Uniform/random mutation is performed within these points.

Before mutation	0	1	1	0	1	1	1	0	0	0
mutation				↑	↑					
After mutation	0	1	1	1	0	1	1	0	0	0

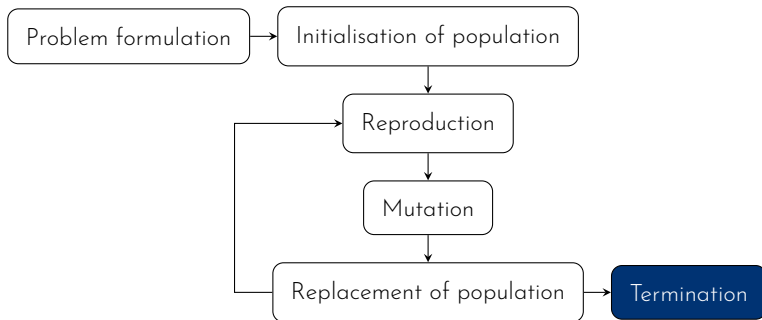
Genetic algorithm



Replacement of Population

1. All children are selected; all parents are discarded; All children are used as the current generation to replace the population.
2. Children and parents are selected based on fitness.

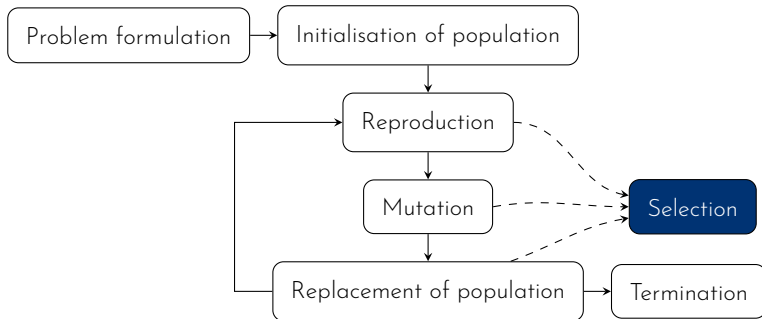
Genetic algorithm

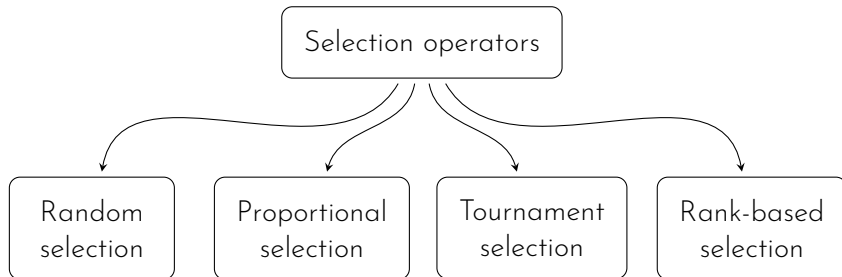


Four common convergence criteria are used as the termination condition.

- ▶ termination due to no improvement is observed over a number of consecutive generations.
- ▶ termination due to no change in population.
- ▶ termination due to finding an acceptable solution.
- ▶ termination due to exceeding a maximum number of iterations.

Genetic algorithm





- ▶ simplest operator
- ▶ each individual has the same probability, i.e. $\frac{1}{n_s}$, where n_s is the population size
- ▶ no fitness information is involved

- uses the fitness of each individual to calculate the probability that the individual is selected

$$P(x_i) = \frac{F(x_i)}{\sum_{j=1}^{n_s} F(x_j)}$$

where,

$P(x)$ is the probability that x is selected

$F(x)$ is the scaled fitness of x

(to have positive value of the fitness)

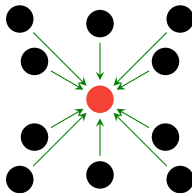
n_s is the total number of individuals in the population

- ▶ selects a group n_{ts} of individuals randomly from the population and pick the best individual (individual with highest fitness)
- ▶ repeats until the desired number of individuals is achieved
- ▶ n_{ts} affects how much the best individual dominates the selection result

- ▶ individuals are ranked according to their fitness (in decreasing order)
- ▶ the probability of selecting an individual is calculated based on its rank (instead of its actual fitness value)

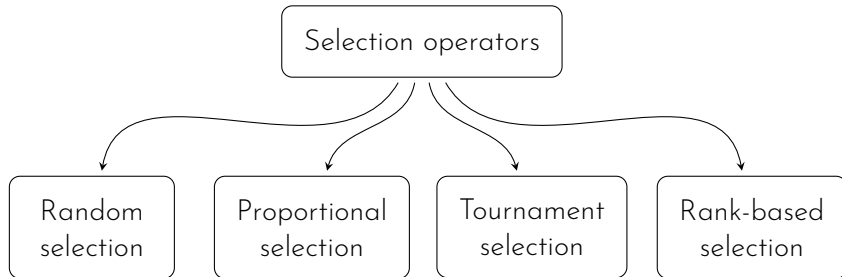
The selection operators are characterised by their **selective pressure**.

Selective pressure is defined as the speed at which the best solution will occupy the entire population by repeated application of the selection operator alone.



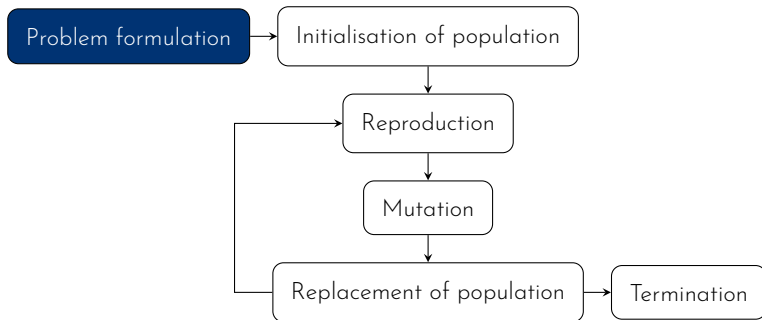
High selective pressure decreases diversity (thus limiting the exploration abilities of the population), may lead to premature convergence to suboptimal solutions.

Low selective pressure increases the time to convergence.



Compare the selective pressure of the different selection operators

Genetic algorithm



SUNWAY UNIVERSITY Jeffrey Cheah Foundation
Nurturing the Seeds of Wisdom

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

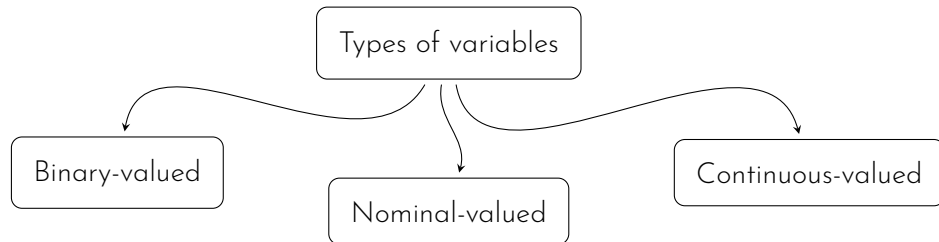
As mentioned,
genetic algorithm is an algorithm to solve an optimisation problem in which we are optimising the values of the chromosome to maximise the fitness of the chromosome.

An individual chromosome is a potential solution for the optimisation problem. The variables to be optimised are genes in the chromosome.

The classic representation scheme for chromosomes are binary vectors with fixed length.

0	1	1	0	1	0
---	---	---	---	---	---

A chromosome can be used to represent different types of variables.



Binary-valued variables

- ▶ boolean data
- ▶ each variable occupies only 1 bit

Example,

- ▶ one binary-valued variable: 0
- ▶ two binary-valued variables: 0 0
- ▶ three binary-valued variables: 0 0 0

Nominal-valued variables

- ▶ categorical data
- ▶ one-hot encoding

Example,

- ▶ a variable containing the information of weather
- ▶ set of possible values: {hot, windy, cloudy, rain, thunderstorm}
- ▶ encoded as a chromosome:

0	1	0	0	0
---	---	---	---	---

A nominal-valued variable with 2 possible states can be represented as a 1-bit binary string.

Is this statement true or false? why?

Continuous-valued variables

- ▶ mapped to a binary string using **binary coding**
 1. defining a finite range for the continuous value, $[x_{\min}, x_{\max}]$,
 2. defining the number of bits for the binary string, n_b , and
 3. convert the continuous value, x from the range of $[x_{\min}, x_{\max}]$ to the value b (rounded to whole number) in the range of $[0, 2^{n_b} - 1]$

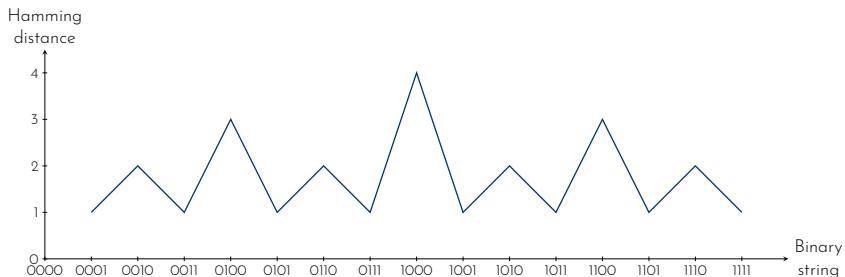
For example, taking the range of 10 to 100, what is the 4-bit binary string representation of the value 16? What is the 5-bit binary string representation? What has changed when the number of bits is changed?

The accuracy of the conversion depends on the number of bits used.

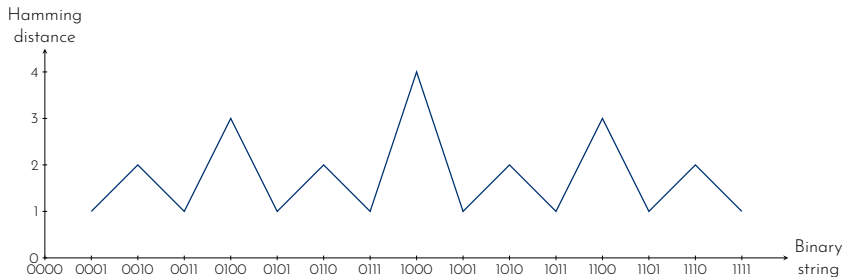
The maximum attainable accuracy of a binary string is

$$\frac{x_{\max} - x_{\min}}{2^{n_b} - 1}$$

Binary coding causes the **hamming cliff**.



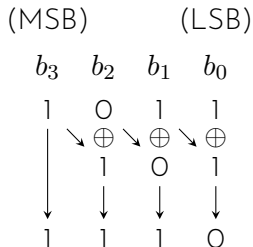
Hamming distance is the number of changing bits going from one binary string to another.



- ▶ **Hamming cliff** is formed when two numerical adjacent values have bit representations that are far apart.
- ▶ Going from one value to the nearest value means modification of multiple bits.
- ▶ This increases the difficulty for the algorithm to identify the optimal solution.

The alternative representation to solve this problem is the **Gray coding**, which makes the adjacent values to have Hamming distance of 1.

Binary-to-Gray code conversion



Maintain the most significant bit (MSB)



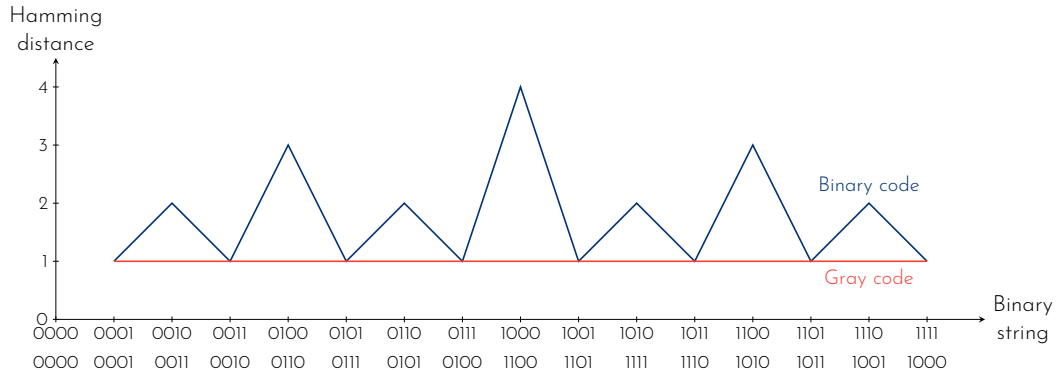
Exclusive OR (X-OR) the current bit with the left bit to get next bit



Continue until all bits are identified

Encoding

Gray code



- ▶ The fitness of an individual represents how well the individual can survive and reproduce.
- ▶ Therefore a fitness function has to be carefully designed for meaningful comparison of fitness between different individuals.
- ▶ The fitness function takes input of a chromosome representation and produces a scalar value which represents the fitness of the chromosome.
- ▶ The higher the fitness value, the higher chance the chromosome will survive and be selected for reproduction.

Example 1

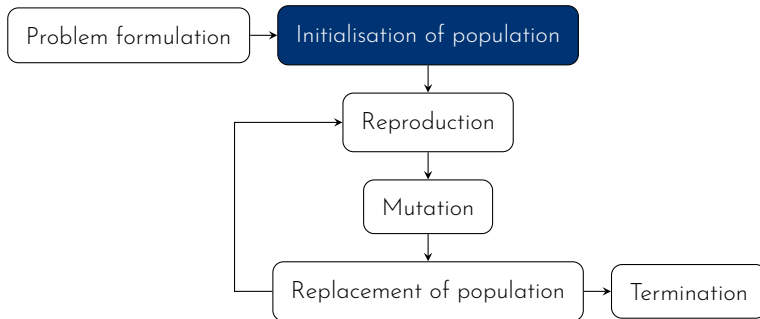
Problem: What is the maximum value of $f(x) = 15x - x^2$, assuming x can only be integers between 0 and 15?

Encoding of solution As x can only be an integer between 0 and 15, we can use a 4-bit binary string to represent the solution.

Integer	Binary code	Gray code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100

Integer	Binary code	Gray code
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Genetic algorithm



Example 1

Initialisation of population

Let's define the population size N to be 10. The initialisation is performed by randomly generating 10 chromosomes from the solution space.

$$v_1 = 0110_g = 4$$

$$v_2 = 1111_g = 10$$

$$v_3 = 0000_g = 0$$

$$v_4 = 1010_g = 12$$

$$v_5 = 0010_g = 3$$

$$v_6 = 1100_g = 8$$

$$v_7 = 1000_g = 15$$

$$v_8 = 1011_g = 13$$

$$v_9 = 0011_g = 2$$

$$v_{10} = 0001_g = 1$$

Can two individuals have the same value?

Example 1

Formulation of fitness function

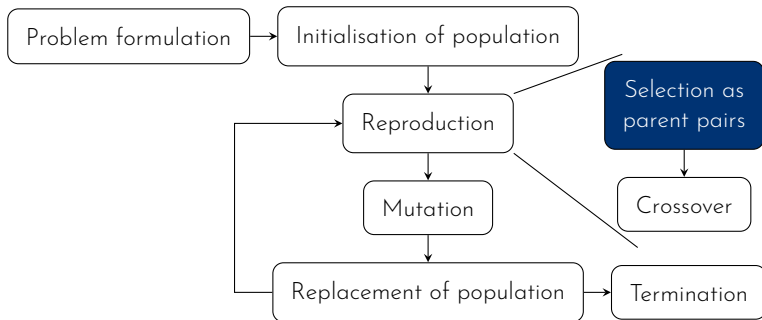
Since we are solving the problem of finding the maximum value of $f(x) = 15x - x^2$, we can use $f(x)$ as our fitness function. This is because, the larger the value of $f(x)$ for a potential solution, the closer the potential solution is towards the desired solution, and thus it should be of higher fitness.

Example 1

Calculating the fitness of each individuals using the fitness function $f(x) = 15x - x^2$

Label	Binary string	Integer value	Fitness
v_1	0110	4	44
v_2	1111	10	50
v_3	0000	0	0
v_4	1010	12	36
v_5	0010	3	36
v_6	1100	8	56
v_7	1000	15	0
v_8	1011	13	26
v_9	0011	2	26
v_{10}	0001	1	14

Genetic algorithm



Example 1

Selection as parent pairs

Using proportional selection, the probability of the selection of the individual is given by the fitness ratio $\frac{f(x_i)}{\sum_{\forall j} f(x_j)}$

Label	Integer value	Fitness	Fitness ratio
v_1	4	44	0.1528
v_2	10	50	0.1736
v_3	0	0	0
v_4	12	36	0.125
v_5	3	36	0.125
v_6	8	56	0.1944
v_7	15	0	0
v_8	13	26	0.0903
v_9	2	26	0.0903
v_{10}	1	14	0.0486

Example 1

Proportional selection is also known as the roulette wheel selection.

Visually we can **arrange the individuals on a roulette wheel** according to their fitness ratios. Then the selection process is carried out by simulating the turning of the roulette wheel.

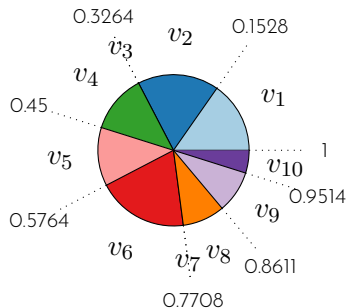
Example 1

Starting from the first individual v_1 , the cumulative fitness ratio for each individual is calculated.

Label	Fitness ratio	Cumulative fitness ratio
v_1	0.1528	0.1528
v_2	0.1736	0.3264
v_3	0	0.3264
v_4	0.125	0.4514
v_5	0.125	0.5764
v_6	0.1944	0.7708
v_7	0	0.7708
v_8	0.0903	0.8611
v_9	0.0903	0.9514
v_{10}	0.0486	1.0

Example 1

Roulette wheel with all individuals



To simulate selecting an individual by turning the roulette wheel,

1. select a random number between 0 and 1, and
2. select the individual within whose area the random number lies.

Example 1

We will be using sexual crossover for reproduction. Therefore to obtain 10 offsprings (population size, $N = 10$), we need 5 parent pairs.

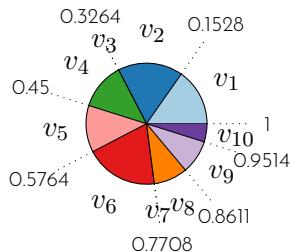
To choose the parent pairs using roulette wheel selection, we will generate 10 random values each lies between 0 and 1 to correspond to each parent.

Label	Parent A	Parent B
p_1	0.9189	0.1049
p_2	0.3305	0.0868
p_3	0.8211	0.9057
p_4	0.5780	0.9635
p_5	0.2476	0.8286

Example 1

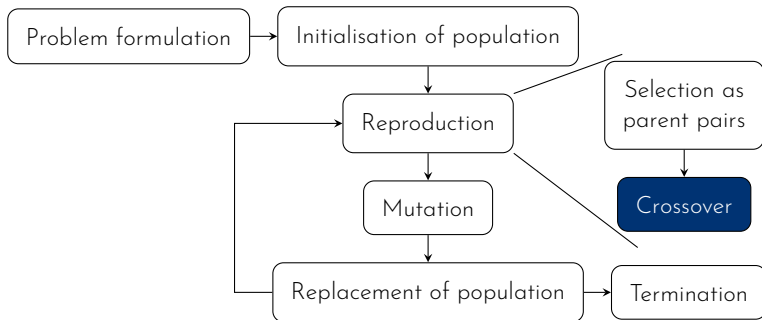
Using the roulette wheel and the cumulative fitness ratios, we can identify the individuals selected as parents.

Label	Parent A		Parent B	
p_1	0.9189	v_9	0.1049	v_1
p_2	0.3305	v_4	0.0868	v_1
p_3	0.8211	v_8	0.9057	v_9
p_4	0.5780	v_6	0.9635	v_{10}
p_5	0.2476	v_2	0.8286	v_8



1. Do we allow one individual being selected in different parent pairs?
2. Do we allow one parent pair consists of the same individual twice?

Genetic algorithm



Example 1

The next question is, how do we perform the crossover? How do we decide the crossover point?

For each pair of parents, we generate a random integer number between 0 and $n - 2$ where n is the length of an individual, and 0 denotes the position of the left-most bit.

The crossover point is the point after the bit denoted by the random integer.

Example 1

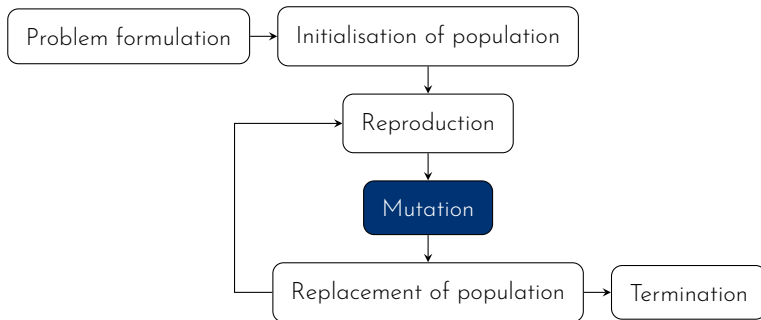
Label		Parent A		Parent B		Random integer	Child 1	Child 2
p_1	v_9	0 0 1-1	v_1	0 1 1-0		2	0010	0111
p_2	v_4	1-0 1 0	v_1	0-1 1 0		0	1110	0010
p_3	v_8	1 0 1-1	v_9	0 0 1-1		2	1011	0011
p_4	v_6	1 1-0 0	v_{10}	0 0-0 1		1	1101	0000
p_5	v_2	1 1-1 1	v_8	1 0-1 1		1	1111	1011

Example 1

Let's label the children

c_1	0010	c_6	0111
c_2	1110	c_7	0010
c_3	1011	c_8	0011
c_4	1101	c_9	0000
c_5	1111	c_{10}	1011

Genetic algorithm



Example 1

Mutation is determined by the mutation rate/probability, p_m . In this case, let us define $p_m = 0.01$.

As every bit has an equal chance of mutation, a random number between 0 and 1 is generated for each bit. If the random number is less than p_m , i.e. 0.01, then the bit will be mutated.

40 random numbers are generated, only the 3rd bit of the 2nd child (c_2) is subjected to mutation.

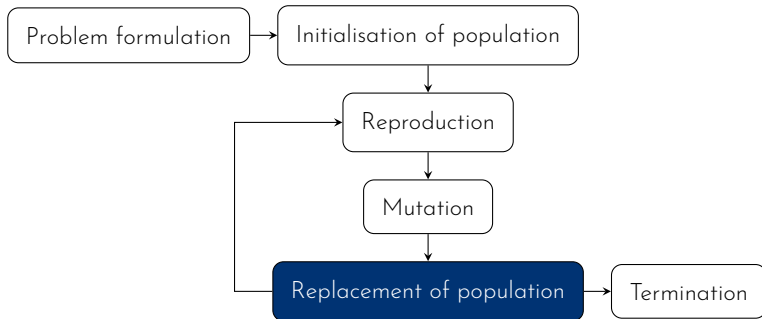
$$c_2 \quad 1\underline{1}10 \rightarrow 1\underline{1}00$$

Example 1

The final versions of the children produced are

Child	Gray code	Decimal value	Fitness
c_1	0010_g	3	36
c_2	1100_g	8	56
c_3	1011_g	13	26
c_4	1101_g	9	54
c_5	1111_g	10	50
c_6	0111_g	5	50
c_7	0010_g	3	36
c_8	0011_g	2	26
c_9	0000_g	0	0
c_{10}	1011_g	13	26

Genetic algorithm



Example 1

If we define that the replacement strategy such that all parents are replaced by their offsprings, the 10 children will form the next generation.

Repeat the process of crossover and mutation until one of the termination criteria is fulfilled.

Example 1

The average fitness of the original generation:

$$\frac{44 + 50 + 0 + 36 + 36 + 56 + 0 + 26 + 26 + 14}{10} = 28.8$$

The average fitness of the next generation:

$$\frac{36 + 56 + 26 + 54 + 50 + 50 + 36 + 26 + 0 + 26}{10} = 36.0$$

The new generation has a higher average fitness compare to the original generation.
The population has, overall, become better.

Can we be sure that, based on the mechanism of genetic algorithm and the synergy of different processes in genetic algorithm, the population is **evolving instead of degenerating**?

Schemata theorem

Schemata theorem shows that the population will contain more individuals with better fitness after multiple generations.

Taking an example of 10 chromosomes with the following fitness values (higher better)

Label, c_i	Chromosome	Fitness, f_i
c_0	0000	4
c_1	0001	10
c_2	0011	18
c_3	0010	24
c_4	0110	28
c_5	0111	30
c_6	0101	30
c_7	0100	28
c_8	1100	24
c_9	1101	18

Focusing on the 2 chromosomes with the highest fitness values, c_5 and c_6 .

0111, 0101

We can generalise the 2 chromosomes as

01*1

where * denotes “don’t care”.

The generalised representation is called a schema (plural: schemata)

Schemata theorem

Label, c_i	Chromosome	Fitness, f_i
c_0	0000	4
c_1	0001	10
c_2	0011	18
c_3	0010	24
c_4	0110	28
c_5	0111	30
c_6	0101	30
c_7	0100	28
c_8	1100	24
c_9	1101	18

Assuming we are using proportional selection, the probability of selecting a chromosome is

$$P(c_i) = \frac{f_i}{\sum_{i=0}^9 f_i}$$

Therefore the probability of selecting either of c_5 and c_6 is

$$P(c_5 \text{ or } c_6) = \frac{f_5 + f_6}{\sum_{i=0}^9 f_i} = \frac{2 \times f_{5/6}}{\sum_{i=0}^9 f_i}$$

where $f_{5/6}$ is the average fitness of chromosomes c_5 and c_6 .

Assuming the generation goes through selections of 10 chromosomes before the next generation, the expected number of times c_5 or c_6 being selected is

$$\begin{aligned}m_{5/6}(t+1) &= 10 \times P(c_5 \text{ or } c_6) \\&= 10 \times \frac{2 \times f_{5/6}}{\sum_{i=0}^9 f_i} \\&= \frac{2 \times f_{5/6}}{f_{\text{avg}}}\end{aligned}$$

where f_{avg} is the average fitness of the whole population

Schemata theorem

$$m_{5/6}(t+1) = \frac{2 \times f_{5/6}}{f_{\text{avg}}}$$

Will $m_{5/6}(t+1) > 2$ or $m_{5/6}(t+1) < 2$?



Let's generalise the equation,

$$m_{5/6}(t+1) = \frac{2 \times f_{5/6}}{f_{\text{avg}}}$$

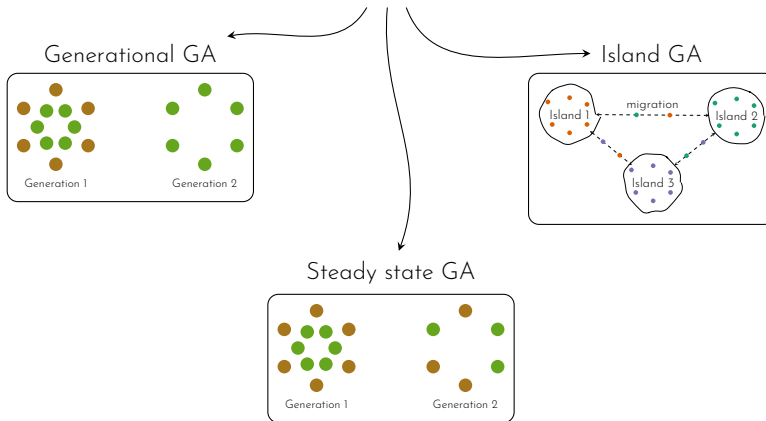
- ▶ $m_{5/6}$ is the number of selected chromosomes with schema 01^*1 , m_H
- ▶ 2 is the number of chromosomes with schema 01^*1 at current generation, $m_H(t)$
- ▶ $f_{5/6}$ is the average fitness of the chromosomes with schema 01^*1 , f_H

$$m_H(t+1) = m_H(t) \cdot \frac{f_H}{f_{\text{avg}}}$$

$$m_H(t+1) = m_H(t) \cdot \frac{f_H}{f_{\text{avg}}}$$

- ▶ What happens to schemata with fitnesses lower than the average fitness?
- ▶ What happens to schemata with fitnesses higher than the average fitness?
- ▶ How would the average fitness of the next generation compare to the previous generation?

Variant on replacement strategy,
i.e. how the offsprings replace the parents.

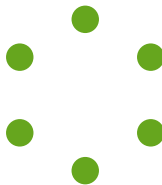


Generational genetic algorithm (GGA)

- ▶ all parents are replaced by their offsprings in the next generation
- ▶ no overlapping between the current and the new population. The overlapping between current and new population is referred to as **generation gap**. GGAs have zero generation gap.



Generation 1



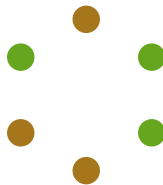
Generation 2

Steady state genetic algorithm (SSGA)

- ▶ once an offspring is created, a decision to whether the parent or the offspring survives to the next generation
- ▶ there is an overlap between the current and the new population. SSGAs have large generation gaps.



Generation 1

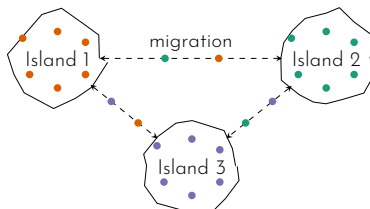


Generation 2

Examples of replacement strategies:

- ▶ replace worst - offspring replaces the worst individual
- ▶ replace random - offspring replaces random individual
- ▶ kill tournament - randomly select a group, offspring replace the worst in the group
- ▶ replace oldest - offspring replace the oldest individual
- ▶ parent-offspring competition - decide if a offspring replaces one of its parents

- ▶ multiple populations
- ▶ each population evolve within the island (selection, crossover, and mutation) independently of other islands
- ▶ migration happens when an individual from an island moves and replaces another individual in another island



CSC3034 Current Course Update

04 Genetic Algorithm - handout

Synchronized from the current CSC3034 course site on 14 August 2026.

- Current Lab 3 covers a complete GA workflow and continuous waypoint encoding.
- The Isaac Sim extension evolves collision-aware mobile-robot trajectories.
- A Matplotlib fallback demonstrates the optimization without simulator hardware.

Current materials author: Dr Tang Tiong Yew

See the synchronized lab notes and runnable examples for full instructions.